

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME:	Cloud Computing and Big Data Analytics	COURSE CODE:	CSA15
COURSE OUTCOME:	CO5: Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN. (BL5)		
WEIGHTAGE / MARKS:	Weightage: 40% Total Marks: 50	TAXONOMY / COUNT:	Dave's Level 4 (Articulation) Q. Count: 5

CODE OF CONDUCT

I **Aakash R / 192524462** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Aakash R**

1. HDFS-based storage solution for a media platform

Constraints:

- Data availability > 99.9%
- Replication factor ≤ 3
- Cost-efficient storage growth

Solution:

Use an HDFS cluster with a replication factor of 3 for important media data. Store files as large HDFS blocks and distribute the blocks across different DataNodes and racks.

Design:

- **NameNode:** Maintains metadata and file directory information.
- **DataNodes:** Store actual media block replicas.
- **Replication factor = 3:** Each block has three copies placed across different DataNodes, preferably spanning different racks.
- **Rack awareness:** Prevents all replicas from being placed within the same rack to guard against switch failures.
- **Compression:** Compress metadata, logs, thumbnails, and supported media formats to reduce storage footprints.
- **Storage tiers:** Frequently accessed media resides on high-performance storage (SSD/Hot), while older archival media moves to cold storage (HDD/Archive).
- **Monitoring:** Continuous heartbeat monitoring of DataNodes to automatically detect failures and trigger re-replication of under-replicated blocks.

Justification: Replication factor 3 provides high availability and fault tolerance while strictly adhering to the constraint. Rack-aware placement protects data availability even during complete rack outages. Compression and tiered storage optimize infrastructure costs as media volume expands.

2. MapReduce workflow for log processing within 20 minutes

Constraints:

- Processing must finish within 20 minutes

- Limited number of cluster nodes

Solution:

Design an optimized MapReduce job with parallel mappers, intermediate combiners, and tuned reducer tasks.

Workflow:

Log Files → Input Splits → Mapper → Shuffle & Sort → Reducer → Output

Task Distribution & Design:

- **Input Splits & Mappers:** Divide log files into optimal input splits; assign one mapper per split. Multiple mappers execute concurrently per node within CPU/memory boundaries.
- **Mapper Processing:** Extract critical fields (IP address, timestamp, URL, HTTP status code, response time) and emit structured key-value pairs (e.g., <IP_Address , 1>).
- **Combiners:** Utilize in-memory combiners at the map phase to perform partial aggregation, significantly minimizing network I/O during shuffle.
- **Intermediate Compression:** Compress map output streams (e.g., using Snappy) to accelerate network transfer.
- **Reducers:** Receive aggregated keys, compute final metrics (traffic counts, error rates, average latencies), and write results to HDFS.

Justification: Massive mapper parallelism cuts execution latency. Combiners and intermediate compression slash network bottlenecks, while balanced reducer partitioning ensures no node becomes a straggler, guaranteeing completion within the 20-minute batch SLA.

3. YARN scheduler policy for ETL, reporting, and ML

Constraints:

- Fair resource sharing
- ETL must receive higher priority
- Recovery from node failure

Proposed Scheduler: Capacity Scheduler

Queue	Priority	Purpose	Suggested Capacity
ETL	High	Data ingestion, cleaning, and transformation	50%
Reporting	Medium	BI dashboards and executive query workloads	25%
Machine Learning	Normal	Model training and advanced exploratory analytics	25%

Policy Configuration:

1. Assign high queue priority to ETL to guarantee timely downstream data readiness.
2. Enforce minimum guaranteed capacities (25% each) for Reporting and ML to avoid starvation.
3. Configure maximum queue capacity limits to prevent any single workload from monopolizing resources.
4. Enable resource elasticity (queue preemption / sharing) so idle capacities are utilized across queues.
5. Enable YARN Application Timeline and Work-Preserving ResourceManager Restart for fast failure recovery.
6. Detect NodeManager crashes via ZooKeeper/heartbeats and automatically reschedule failed task containers on healthy nodes.

Justification: Capacity Scheduler satisfies strict priority requirements for ETL while ensuring deterministic fair shares for Reporting and ML. Built-in container reassignment provides seamless fault tolerance against node failures.

4. Enterprise data ingestion from SAN, NAS, and databases

Constraints:

- Reliable backup
- Minimal duplication
- Multiple heterogeneous data sources

Ingestion Architecture:

SAN / NAS / Operational DBs → Apache NiFi (Validation + Deduplication) → HDFS Data Lake → Backup / DR
→ Analytics & ML

Components & Strategy:

- **SAN/NAS Ingestion:** Use NiFi GetFile/TailFile processors and storage connectors to monitor directories and ingest files incrementally.
- **Database Ingestion:** Employ Change Data Capture (CDC) or incremental JDBC querying based on timestamps/primary keys to ingest changed rows rather than executing full table dumps.
- **Apache NiFi Pipeline:** Centrally orchestrates dataflows, enforces back-pressure flow control, handles schema transformations, and routes data securely.
- **Deduplication Mechanism:** Compute MD5/SHA-256 hashes of record contents or file metadata. Check against a distributed cache (e.g., Redis / NiFi DistributedMapCache) prior to lake ingestion.
- **HDFS Storage & Backup:** Store data centrally with replication factor 3. Maintain off-site automated snapshots to an isolated DR cluster or cloud cold tier.

Justification: CDC and hashing-based deduplication eliminate duplicate ingestion and redundant storage. Centralized NiFi routing ensures robust throughput, while multi-tier replication guarantees backup reliability.

5. End-to-End Hadoop ecosystem solution

Constraints:

- End-to-end security
- Scalability
- High availability (HA)
- Seamless integration with Apache NiFi / ETL

Architecture & Subsystems:

- **Data Ingestion (Apache NiFi):** Connects to databases, streaming endpoints, logs, and SAN/NAS storage. Provides back-pressure regulation and ETL transformation prior to loading.
- **Security Framework:**
 - *Authentication:* Kerberos/SPNEGO across all cluster services.
 - *Authorization & Governance:* Apache Ranger for fine-grained RBAC and Apache Atlas for metadata/lineage tracking.
 - *Encryption:* TLS/SSL for wire encryption (RPC, HTTP, Shuffle); HDFS Transparent Encryption (KMS) for data-at-rest.
- **Storage Layer (HDFS):** Active/Standby NameNodes with ZooKeeper Failover Controller (ZKFC) and Quorum Journal Nodes (QJM) ensuring zero single point of failure; rack-aware replication factor 3.
- **Resource Management (YARN):** High Availability ResourceManagers with Capacity Scheduling to scale compute across multi-tenant workloads.
- **Processing Engines:** MapReduce for legacy batch processing; Apache Spark for high-speed in-memory analytics/ML; Apache Hive with LLAP for low-latency interactive SQL querying.

- **High Availability Summary:** Active/Standby NameNodes, dual YARN ResourceManagers, multi-node DataNode fault-tolerance, and automatic container retry.

Justification: This unified architecture achieves comprehensive enterprise security via Kerberos and Ranger, seamless scale through YARN/HDFS clustering, zero downtime through HA NameNode/ZKFC configurations, and streamlined ingestion with Apache NiFi.